

```
import pandas as pd

# Define FAQ data
faq_data = [
    {"question": "What is your return policy?", "answer": "Our return policy allows returns within 30 days of purchase."},
    {"question": "How can I contact customer support?", "answer": "You can contact customer support via email at support@example.com or call 1-800-123-4567."},
    {"question": "What are your business hours?", "answer": "Our business hours are Monday to Friday, 9 AM to 5 PM EST."},
    {"question": "Where can I find my order status?", "answer": "You can check your order status by logging into your account on our website."},
    {"question": "Do you offer international shipping?", "answer": "Yes, we offer international shipping to most countries."},
    {"question": "Can I change or cancel my order?", "answer": "Order changes or cancellations are accepted up to 48 hours before shipping."},
    {"question": "What payment methods do you accept?", "answer": "We accept major credit cards (Visa, MasterCard, American Express) and PayPal."},
    {"question": "How long does shipping take?", "answer": "Standard shipping typically takes 5-7 business days."},
    {"question": "Do you have a loyalty program?", "answer": "Yes, we have a loyalty program that rewards frequent customers."},
    {"question": "How do I reset my password?", "answer": "You can reset your password by clicking the 'Forgot Password' link on the login page."}
]

# Create a DataFrame
df_faqs = pd.DataFrame(faq_data)

display(df_faqs.head())
```

	question	answer	
0	What is your return policy?	Our return policy allows returns within 30 day...	
1	How can I contact customer support?	You can contact customer support via email at ...	
2	What are your business hours?	Our business hours are Monday to Friday, 9 AM ...	
3	Where can I find my order status?	You can check your order status by logging int...	
4	Do you offer international shipping?	Yes, we offer international shipping to most c...	

```
# Install NLTK if not already installed
!pip install nltk

import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
import string

# Download necessary NLTK data
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.4.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.0)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2024.11.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
import nltk
nltk.download('punkt_tab')

# Initialize lemmatizer and stopwords
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))
```

```
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))
    # Tokenize
    tokens = word_tokenize(text)
    # Remove stopwords and lemmatize
    processed_tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in stop_words]
    return ' '.join(processed_tokens)

# Apply preprocessing to the 'question' and 'answer' columns
df_faqs['processed_question'] = df_faqs['question'].apply(preprocess_text)
df_faqs['processed_answer'] = df_faqs['answer'].apply(preprocess_text)

display(df_faqs[['question', 'processed_question', 'answer', 'processed_answer']].head())
```

[nltk_data] Downloading package punkt_tab to /root/nltk_data...

[nltk_data] Unzipping tokenizers/punkt_tab.zip.

	question	processed_question	answer	processed_answer
0	What is your return policy?	return policy	Our return policy allows returns within 30 day...	return policy allows return within 30 day purc...
1	How can I contact customer support?	contact customer support	You can contact customer support via email at ...	contact customer support via email supportexam...
2	What are your business hours?	business hour	Our business hours are Monday to Friday, 9 AM ...	business hour monday friday 9 5 pm est
3	Where can I find my order status?	find order status	You can check your order status by logging int...	check order status logging account website nav...
	~	~	..	~

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Initialize TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()

# Fit and transform the processed FAQ questions
faq_vectors = tfidf_vectorizer.fit_transform(df_faqs['processed_question'])

# Sample user query
user_query = "I want to return an item, what is the policy?"

# Preprocess the user query
processed_user_query = preprocess_text(user_query)

# Transform the preprocessed user query into a TF-IDF vector
user_query_vector = tfidf_vectorizer.transform([processed_user_query])

# Calculate cosine similarity between the user query and all FAQ questions
similarity_scores = cosine_similarity(user_query_vector, faq_vectors)

# Get the index of the most similar FAQ
most_similar_faq_index = similarity_scores.argmax()

# Get the most similar FAQ question and its answer
most_similar_question = df_faqs.loc[most_similar_faq_index, 'question']
most_similar_answer = df_faqs.loc[most_similar_faq_index, 'answer']
max_similarity_score = similarity_scores[0, most_similar_faq_index]

print(f"User Query: {user_query}")
print(f"\nMost Similar FAQ Question: {most_similar_question}")
```

```
print(f"Answer: {most_similar_answer}")
print(f"Similarity Score: {max_similarity_score:.4f}")
```

User Query: I want to return an item, what is the policy?

Most Similar FAQ Question: What is your return policy?

Answer: Our return policy allows returns within 30 days of purchase with a valid receipt.

Similarity Score: 1.0000

```
def get_chatbot_response(user_input, df, tfidf_vectorizer):
    # Preprocess the user query
    processed_user_input = preprocess_text(user_input)

    # Transform the preprocessed user query into a TF-IDF vector
    user_input_vector = tfidf_vectorizer.transform([processed_user_input])

    # Calculate cosine similarity between the user query and all FAQ questions
    similarity_scores = cosine_similarity(user_input_vector, tfidf_vectorizer.transform(df['proce

    # Get the index of the most similar FAQ
    most_similar_faq_index = similarity_scores.argmax()

    # Get the most similar FAQ question and its answer
    most_similar_question = df.loc[most_similar_faq_index, 'question']
    most_similar_answer = df.loc[most_similar_faq_index, 'answer']
    max_similarity_score = similarity_scores[0, most_similar_faq_index]

    print(f"You asked: {user_input}")
    print(f"\nI found the most similar FAQ: '{most_similar_question}' (Similarity: {max_similarity_score:.4f})")
    print(f"Answer: {most_similar_answer}")

# Example of using the chatbot function
user_question = "What are your working hours?"
get_chatbot_response(user_question, df_faqs, tfidf_vectorizer)
```

You asked: What are your working hours?

I found the most similar FAQ: 'What are your business hours?' (Similarity: 0.7071)

Answer: Our business hours are Monday to Friday, 9 AM to 5 PM EST.

```
from IPython.display import display, HTML
import ipywidgets as widgets

def chat_interface(question):
    if question:
        get_chatbot_response(question, df_faqs, tfidf_vectorizer)
    else:
        print("Please type a question.")

# Create a text input widget
question_input = widgets.Text(
    value='',
    placeholder='Type your question here...',
    description='Your Question:',
    disabled=False
)

# Create an output widget to display the chat response
output_area = widgets.Output()

def on_submit(sender):
    with output_area:
        output_area.clear_output()
        chat_interface(question_input.value)

question_input.on_submit(on_submit)
```

```
print("Welcome to the FAQ Chatbot! Ask me anything about our products and services.")  
display(question_input, output_area)
```

Welcome to the FAQ Chatbot! Ask me anything about our products and services.

Your Quest...